

11-1K Algorithmus — Methodensignatur & Prüfungstipps

Zusammenfassung für die IHK-Abschlussprüfung – Methodensignatur lesen, von UML nach Java übersetzen und typische Prüfungsfallen vermeiden

1. Was ist eine Methodensignatur?

Grundform (Java):

Sichtbarkeit static? Rückgabetyt Methodenname(Parameterliste)

امضای متد (Methodensignatur) در جاوا از این بخش‌ها ساخته می‌شود:

`public static int berechneSumme(int[] werte)`

- **Sichtbarkeit** (مثلاً `public` ، `private`)

- گزینهٔ اختیاری `static`

- **Rückgabetyt**: نوع مقداری که برگردانده می‌شود (مثلاً `void` ، `int` ، `boolean`)

- **Methodenname**: مثلاً `berechneSumme`

- **Parameterliste**: نوع + نام ورودی‌ها (مثلاً `int[] werte`)

Beispiel:

```
public static int berechneSumme(int[] werte) {  
    int summe = 0;  
    for (int w : werte) {  
        summe = summe + w;  
    }  
    return summe;  
}
```

در این مثال از روی امضا می‌توانی فوراً بفهمی:

- ورودی: آرایه‌ای از اعداد صحیح (`int[] werte`)

- خروجی: یک عدد صحیح (`int`) که در نهایت باید برگردانده شود

- پس در بدنهٔ متد باید جمع مقادیر را حساب کرده و برگردانی.

2. UML-Notation und Übersetzung nach Java

2.1 UML-Schreibweise

```
+ berechneSumme(werte : int[]) : int
- ermittleDurchschnitt(summe : int, anzahl : int) : double
```

قواعد اصلی در UML:

- + یعنی **public**

- - یعنی **private**

- بعد از نام متغیر یا متد علامت : می‌آید و نوع (Typ) نوشته می‌شود

- در پایان امضا در UML، نوع بازگشتی بعد از : می‌آید

UML → Java 2.2

UML: + berechneSumme(werte : int[]) : int

Java:

```
public int berechneSumme(int[] werte)
```

تبدیل همیشه مستقیم است:

- + → public

- نام متد همان است

- پارامترها: Typ : name در UML → Typ name در Java

- نوع بازگشتی بعد از : در UML → جلوی امضا در Java

3. Typische Methodensignaturen in der IHK-Prüfung

3.1 Suchen (Search)

```
boolean istVorhanden(int[] werte, int gesucht)
```

```
int sucheIndex(String[] namen, String gesucht)
```

این نوع امضا نشان می‌دهد که باید در آرایه/لیست بگردی و یا فقط بگویی عنصر هست/نیست (boolean) یا جایگاهش را برگردانی (int)، معمولاً اندیس یا -1 اگر پیدا نشد).

3.2 Zählen (Count)

```
int zaehlePositive(int[] werte)
```

```
int zaehleKundenMitRabatt(List<Kunde> liste)
```

در این نوع متدها باید یک شمارندهٔ عدد صحیح بسازی، روی تمام عناصر حلقه بروی، شرط را چک کنی و در صورت برآورده شدن شرط، شمارنده را یکی زیاد کنی و در آخر برگردانی.

3.3 Berechnen (Calculate)

```
double berechneDurchschnitt(int[] werte)
```

```
double berechneRabatt(double betrag, double prozent)
```

اینجا امضا به تو می‌گوید که خروجی یک عدد اعشاری است؛ پس معمولاً باید جمع بگیری، تقسیم کنی، درصد بگیری یا فرمول داده‌شده را پیاده‌سازی کنی.

Filtern & Ausgabe 3.4

```
List<Kunde> filterePremiumKunden(List<Kunde> alle)
void druckeKunden(List<Kunde> liste)
```

- اگر خروجی یک `List<...>` باشد، یعنی باید لیست جدید بسازی و فقط عناصر `符合` شرط را به آن اضافه کنی.
- اگر خروجی `void` باشد، یعنی کاری مثل چاپ، ذخیره یا نمایش انجام می‌دهی و چیزی برنمی‌گردانی.

4. Prüfungs-Tipps: Signatur lesen und Problem verstehen

4.1 Tipp 1 – Erst den Rückgabetyt analysieren

- `boolean` → پاسخ باید «درست/نادرست» باشد؛ الگوریتم معمولاً شامل یک شرط است که اگر برآورده شود، `true` برمی‌گرداند، وگرنه در پایان `false`.
- `int / double` → باید یک مقدار عددی محاسبه کنی؛ جمع، تعداد، میانگین، حداکثر، حداقل.
- `String` → ساخت یا ترکیب متن؛ مثلاً نام کامل، پیام، یا توضیح.
- `void` → متد فقط اثر جانبی دارد (چاپ، ذخیره، تغییر وضعیت)، بدون مقدار بازگشتی.

4.2 Tipp 2 – Parameterliste = Eingaben der Aufgabe

- هرجا در صورت سؤال کلمه‌هایی مثل «Input», «Eingabe», «Gegeben ist ...» دیدی، همان چیزها باید در Parameterliste ظاهر شوند.
 - مثال: اگر در متن گفته شده «آرایه‌ای از مقادیر و یک Grenzwert داده شده است»، امضا باید چیزی مانند زیر باشد:
- ```
public static int zaehleWerteGroesser(int[] werte, int grenze)
```

### 4.3 Tipp 3 – Methodenname genau lesen

- نام متد تقریباً همیشه وظیفه را لو می‌دهد:
- `berechneSumme` → جمع بگیر
- `ermittleMax` → حداکثر را پیدا کن
- `istPositiv` → مقدار `boolean` برمی‌گرداند
- `druckeRechnung` → فقط چاپ یا نمایش
- در امتحان، خیلی وقت‌ها اگر نام متد را دقیق بخوانی، نیمی از راه حل را فهمیده‌ای.

## 5. Musterlösungen: typische IHK-Algorithmen

### 5.1 boolean enthaelt (...)

```
public static boolean enthaelt(int[] werte, int gesucht) {
 for (int w : werte) {
```

```

 if (w == gesucht) {
 return true;
 }
 }
 return false;
}

```

الگوی کلاسیک جستجو:

- روی همهٔ عناصر آرایه حلقه می‌روی
- اگر عنصر برابر مقدار موردنظر بود → بلافاصله true برگردان
- اگر حلقه تمام شد و چیزی پیدا نشد → در انتها false برگردان

## 5.2 int sucheIndex (...)

```

public static int sucheIndex(String[] namen, String gesucht) {
 for (int i = 0; i < namen.length; i++) {
 if (namen[i].equals(gesucht)) {
 return i;
 }
 }
 return -1;
}

```

این الگوریتم همان جستجو است، اما به جای true/false، اندیس عنصر را برمی‌گرداند. اگر پیدا نشد، مقدار قراردادی -1 برگردانده می‌شود.

## 5.3 double berechneDurchschnitt (...)

```

public static double berechneDurchschnitt(int[] werte) {
 int summe = 0;
 for (int w : werte) {
 summe = summe + w;
 }
 if (werte.length == 0) {
 return 0.0;
 }
 return (double) summe / werte.length;
}

```

این همان الگوی «جمع + تقسیم» است:  
 - یک متغیر summe برای جمع مقادیر

- حلقه روی آرایه
- بررسی مورد خاص آرایه خالی
- تقسیم جمع بر تعداد عناصر و برگرداندن مقدار اعشاری

## 6. Von Signatur zum Algorithmus in 3 Fragen

Beispielsignatur:

```
public static int zaehlePositive(int[] werte)
```

برای طراحی سریع الگوریتم از روی امضا، همیشه این سه سؤال را از خودت بپرس:

۱. ورودی چیست؟  $\rightarrow$  `int[] werte` ، پس باید روی تمام عناصر حلقه بروم.
  ۲. خروجی چیست؟  $\rightarrow$  `int` ، پس باید یک شمارنده داشته باشم و در پایان آن را برگردانم.
  ۳. شرط یا منطق چیست؟  $\rightarrow$  «عدد مثبت است» یعنی  $w > 0$  .
- الگوریتم:

```
public static int zaehlePositive(int[] werte) {
 int anzahl = 0;
 for (int w : werte) {
 if (w > 0) {
 anzahl = anzahl + 1;
 }
 }
 return anzahl;
}
```

## 7. Typische Aufgabentypen in der Prüfung

### 7.1 Lückentext in der Signatur

جاهای خالی مثل:

```
public static __ sucheIndex(__ namen, String gesucht)
```

باید از روی متن سؤال تشخیص دهی که مثلاً خروجی عدد صحیح است ( `int` ) و ورودی آرایه‌ای از رشته‌هاست ( `[]String` ).

### 7.2 "Welche Signatur passt?" (Mehrfachauswahl)

چند امضا داده می‌شود و باید درست را انتخاب کنی. روش سیستماتیک:  
- آیا Rückgabetyپ با توضیح «Ergebnis» در صورت سؤال همخوانی دارد؟

- آیا تعداد پارامترها و نوع آنها با «Gegeben» در صورت سؤال همخوانی دارد؟
- آیا نام متد با عملکرد موردنظر متن تناسب دارد؟

### UML → Java 7.3

مستقیم طبق قاعده  $-/+$  و `name : Typ` به Java تبدیل می‌کنی. این نوع سؤال معمولاً ساده‌ترین نوع است اگر قواعد را حفظ باشی.

### Text → Signatur 7.4

متن می‌گوید: «متدی بنویسید که بررسی کند آیا عددی در Feld وجود دارد و true یا false برگرداند.»  
امضای درست چیزی شبیه این است:

```
public static boolean enthaelt(int[] werte, int gesucht)
```

## 8. Kurz-Zusammenfassung für die Prüfung

- اول Rückgabetyp را بخوان: boolean → تصمیم بله/خیر، int/double → عدد، void → فقط اثر جانبی.
- هر چیزی که در صورت سؤال به‌عنوان «داده ورودی» آمده باید در Parameterliste باشد.
- هر چیزی که به‌عنوان «نتیجه» توصیف شده باید با نوع بازگشتی و return همخوانی داشته باشد.
- UML را مکانیکی ترجمه کن:  $-/+$  → `name : Typ → Typ` ، public/private .
- الگوهای کلاسیک (جستجو، شمارش، میانگین‌گیری) را از بر کن و در هر سؤال مشابه همان‌ها را اعمال کن.

## 9. Prüfungstipp & Zusammenfassung – Profi-Level

ترفندهای اضافه برای حل سریع‌تر در امتحان:

- اگر در صورت سؤال کلمه‌های `prüfen, enthalten, ist` دیدی و `boolean = Rückgabetyp` بود، تقریباً همیشه الگوی «حلقه + `true / false` + `if`» است.
- اگر خروجی `int` است و در متن از *Anzahl* یا «wie viele» صحبت شده، فوراً یک متغیر شمارنده تعریف کن و در حلقه فقط در صورت برآورده شدن شرط، آن را یکی زیاد کن.
- اگر در UML یا متن کلیدواژه‌های *Array, Feld, Liste* دیدی، یعنی حتماً باید از حلقه (for/while/foreach) استفاده کنی؛ الگوریتم بدون حلقه تقریباً غلط است.
- در سؤالات چندگزینه‌ای برای Signatur، گزینه‌ای که `Typ` پارامترهایش دقیقاً مطابق «Gegeben» در متن است، معمولاً پاسخ درست است.
- وقتی `Typ` پارامترها خیلی پیچیده یا غیرمنطقی است (مثلاً `int name` یا `String betrag`)، این گزینه تقریباً همیشه غلط است.
- اگر در Aufgabe گفته شده «Das Ergebnis soll auf dem Bildschirm ausgegeben werden» و هیچ سخنی از "zurückgeben" نیامده، اغلب جواب یک متد `void` است که فقط `System.out.println(...)` استفاده می‌کند.
- اگر *UML-Methodenname mit get beginnt* (مثل `getAnzahl`)، معمولاً `Rückgabetyp` عددی است و پارامتر ندارد؛ اگر گزینه‌ای پارامتر بی‌دلیل داشت، مشکوک است.
- اگر متن از «Durchschnitt, Mittelwert» حرف می‌زند، حواست باشد که تقسیم باید روی یک `double` انجام شود؛ یعنی یک `cast` به `double` لازم است، نه تقسیم صحیح.

- در الگوریتم‌های جستجو همیشه فکر کن: «اگر پیدا نشد چه می‌شود؟» مقدار قراردادی مثل 1- یا false را در انتها فراموش نکن.
- موقع نوشتن Pseudocode، لازم نیست Syntax جاوا ۱۰۰٪ درست باشد؛ مهم این است که ساختار منطقی (حلقه، شرط، شمارنده، مقدار برگشتی) واضح و کامل باشد.
- در صورت سؤال اگر Beispiel-Ausgabe داده شده، حتماً نوع داده و شکل خروجی را از روی آن چک کن؛ این کمک می‌کند که Rückgabetyپ یا void بودن را سریع تشخیص دهی.
- قبل از شروع نوشتن الگوریتم، فقط با نگاه کردن به Methodensignatur سعی کن در یک جمله ساده فارسی برای خودت بگویی این متد چه کار باید بکند؛ اگر این جمله شفاف باشد، پیاده‌سازی کد بسیار ساده‌تر می‌شود.